

Funkcje – przekazywanie tablic jako parametrów

Aby przekazać tablicę do funkcji, celem wykonania operacji na jej komórkach nie trzeba przekazywać wartości każdej z komórek z osobna (nawet nie powinno się tak robić). Należy wykorzystać to, że kolejne komórki tablicy **leżą po kolei obok siebie w pamięci**.

Wystarczy, jeśli jako parametr przekazemy lokalizację w pamięci (adres/wskaźnik) do wybranej przez nas komórki, od której chcielibyśmy zacząć operacje (nie musi to być pierwsza komórka w tablicy).

Dobrym pomysłem jest także przekazanie dodatkowego parametru, jako rozmiaru (ilości komórek), jeśli będziemy wykonywać jakieś operacje z użyciem pętli.

```
void Funkcja_z_tablica_wypelnij( int *Tab, const int rozmiar ) { for( int i = 0; i < rozmiar; i++ )  
{  
    Tab[ i ] = i * 10; // Przykładowe wartości na wypełnienie tablicy }  
} //-----
```

```
void Funkcja_z_tablica_wyswietl( int Tab[], const int rozmiar ) { for( int i = 0; i < rozmiar; i++ )  
{  
    cout.width( 4 );  
    cout << Tab[ i ];  
}  
cout << endl;  
}
```

Funkcja main() wywołująca te funkcje na następnej stronie.

Rozmiar jako parametr:

Zaleca się na zmiennej odpowiedzialnej za rozmiar zastosowanie modyfikatora `const`. Dzięki temu zabezpieczymy sobie „rozmiar” przed modyfikacją w trakcie działania (celowo bądź przypadkowo).

Deklaracja tablicy jako parametru funkcji:

Dane są dwie funkcje jak poniżej:

```
void Funkcja_z_tablica_wypelnij( int *Tab, const int rozmiar )  
void Funkcja_z_tablica_wyswietl( int Tab[], const int rozmiar )
```

Widzimy, że inaczej jest zapisany pierwszy parametr, odpowiedzialny za adres tablicy w pamięci.

Pierwsza funkcja, z użyciem klasycznej deklaracji wskaźnikowej (*). **Druga funkcja**, z użyciem deklaracji wskaźnika na tablicę ([]). Od strony technicznej **nie ma żadnej różnicy** pomiędzy dwoma powyższymi zapisami. Obie deklaracje wskazują na to samo miejsce w pamięci. (oczywiście o ile obydwie wywołania funkcji będą miały taki sam parametr). Chodzi tylko o wygodę w

czytaniu kodu przez programistę.

Zapis z użyciem klamer ([]) widziany z daleka, od razu wyraźnie sugeruje, że mamy do czynienia z grupą elementów (tablicą), natomiast zwykły zapis wskaźnika (*) może sugerować zarówno tablicę, jak i tylko jeden element. Zaleca się jednak stosować zapis z użyciem klamer ([]).

```
int main()
{
    // Tylko jeśli zmienna odpowiedzialna za rozmiar jest typu const (czytaj: // znamy jej wartość w momencie
    // kompilacji) możemy zadeklarować tablicę // statyczną z jej użyciem.
    // W innym wypadku należałoby użyć tablicy dynamicznej.
    const int R = 16;
    int Tablica[ R ];

    // Przekazanie całej tablicy jako parametr funkcji. Nazwa tablicy. Funkcja_z_tablica_wypelnij(
    Tablica, R);

    // Przekazanie całej tablicy jako parametr funkcji. Wskaźnik do pamięci // na jej pierwszy element (o
    indeksie 0).
    // Gdy jako parametr funkcji przekazujemy pierwszy element tablicy // (element o indeksie 0), nie ma
    żadnej różnicy czy parametrem będzie // sama nazwa tablicy, czy czy wskaźnik na jej pierwszy element. //
    Oba zapisy odnoszą się do tej samej lokalizacji w pamięci. // Zaleca się jednak stosowanie tego drugiego
    zapisu (łatwiej zapamiętać // jeden sposób niż dwa różne).
    Funkcja_z_tablica_wyswietl( &Tablica[ 0 ], R );

    cout << endl; // Pusta linia, aby oddzielić wydruki od siebie.

    // Przykład wywołania funkcji tylko dla fragmentu tablicy. W takim wypadku // musimy zastosować
    wywołanie & oraz indeks. Jako element początkowy // przekazujemy (przykładowo) adres komórki o
    indeksie 5. // Jako rozmiar przekazujemy (przykładowo) połowę oryginalnego rozmiaru. // Uwaga!
    Wewnątrz funkcji ten fragment będzie widziany jako osobna // tablica, gdzie jej komórki będą indeksowane
    od zera.
    // Oczywiście wszystkie operacje będą wykonywane na oryginalnej tablicy, // więc operacje na komórce
    [0] będą w rzeczywistości operacjami // na komórce [5] oryginalnej tablicy, operacje na komórce [1] będą
    // w rzeczywistości operacjami na komórce [6] oryginalnej tablicy itd. // Działanie takie prezentuje
    schemat poniżej.
    // Tablica oryginalna, kolor niebieski. Tablica „wewnątrz, jaką widzi // funkcja”, kolor żółty.
    Funkcja_z_tablica_wyswietl( &Tablica[ 5 ], R / 2 );

    return 0;
}
```